Inferno: Adaptive LLM Inference Serving

CUDA kernel · MPI request router · rigorous GPU benchmarking

B.Tech (AIDS)

UG-III (V-Semester)

High-Performance Computing Project · Mid-Term Evaluation 2026

Subject Coordinator

Mr. Suraj Singh

Presented By:

Likhith (DL.AI.U4AID24117)
Karthikeya (DL.AI.U4AID24020)
Sandeep (DL.AI.U4AID24143)
A. Karthik (DL.AI.U4AID24104)

AMRITA VISHWA VIDYAPEETHAM, DELHI NCR, FARIDABAD

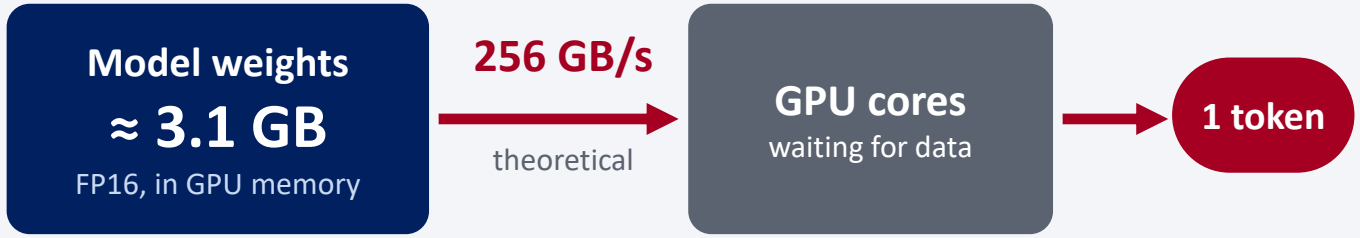
Department of Artificial Intelligence and Data Science

CONTENTS

1. Problem Statement
2. Literature Review
3. Objectives of the project
4. Proposed Methodology
5. Tools and Technologies
6. Experimental setup
7. Results
8. Challenges
9. Contributions, Conclusions, Plan & Team
10. References

Problem Statement

Why is generating text slow? A memory problem.



Measured FP16 baseline: 67 tokens/s (vLLM 0.11, batch 1, measured)

Weight-read bandwidth estimate: A 3.1 GB FP16 model at 256 GB/s theoretical memory bandwidth gives an idealized weight-read time of approximately 12 ms per token, corresponding to an upper-bound estimate of approximately 83 tokens/s under batch-1, weight-bound assumptions. Actual throughput depends on kernel efficiency, compute, and serving overhead.

1 Memory-bound decoding

At small batch sizes, speed is often limited by the bytes read per token. Smaller (int4) weights → fewer bytes.

2 Bursty traffic, tail latency

Production traffic is bursty. Users feel the slowest requests (p99), not the average.

3 Coordinating replicas

Serving several model replicas needs a low-overhead layer to route independent requests (MPI).

Our question

Can a parallel, adaptive serving stack beat fixed configurations – and why?

Why HPC?

GPU / CUDA – billions of independent multiply-adds
MPI – coordinating independent requests
Rigorous benchmarking – to trust any speed claim

Literature Review

Four research threads our project builds on — and the gap between them

Continuous & Paged Serving

- **Orca [1]** iteration-level (continuous) batching
- **vLLM [2]** PagedAttention — paged KV-cache memory

Disaggregated Serving

- **DistServe [3]** splits prefill / decode across GPUs
- **Splitwise [4]** phase-specialized GPU pools

Quantization & Kernels

- **AWQ [5], GPTQ [6]** accurate 4-bit weights
- **Marlin [7]** fast 4-bit GPU kernel — our baseline

Rigorous Benchmarking

- **Hoefler & Belli [8]** 12 rules for HPC benchmarking
- **Georges et al. [9]** statistically sound perf. claims

The gap we fill:
none of these study the kernel, the router and the statistics TOGETHER on one shared GPU — with paired tests and negative results reported honestly.

Objective of the Project

3 · OBJECTIVES

1	Fused int4 CUDA kernel reaching ≥ 85% of theoretical DRAM bandwidth (effective)	Achieved: 87–95% of peak
2	Integrate the kernel into vLLM; measure real decode speed	Done: parity with Marlin at 1 sequence
3	Test adaptive routing against the strongest fixed baselines (pre-specified H1)	Done: rejected on 1 GPU · multi-GPU pending
4	MPI request router: measure overhead and scaling	1 GPU done · multi-GPU PLANNED
5	Rigorous, reproducible method – every number from raw data	Ongoing: 81 tests passed, audit trail

Proposed Methodology: Workflow

1 Workload Generator
Poisson · bursty · mixed lengths; seeded

2 Open-loop Client
sends on schedule; records TTFT, TPOT, p99

3 Request Router
policy · HTTP baseline · MPI dispatcher (rank 0)

4 vLLM Replicas
fast tier + bulk tier sharing one GPU

5 Fused int4 CUDA Kernel
decode weights, multiply, reduce – on the GPU

6 Metrics & Power
latency, goodput, tokens/s, watts (every 200 ms)

7 Statistics
paired t-test + Wilcoxon, Holm, effect size

8 Report & Dashboard
every number generated from raw logs

Inputs | System under test | Measurement & analysis

Key idea: replay the SAME seeded traffic through every policy – differences come from the policy, not luck.

Proposed Methodology: Parallelization

	What is independent?	How it is parallelized	Why this framework?
LEVEL 1 CUDA inside one layer	each output row of the matrix · each row = K/32 chunks of 32 weights	16 lanes per row · 128-bit coalesced loads · warp-shuffle reduction	fine-grained data parallelism on a memory-bound GPU
LEVEL 2 vLLM across users	tokens of different users' requests	continuous batching · one decoded weight reused for up to B vectors	production scheduler – not re-invented
LEVEL 3 MPI across replicas	whole requests → independent servers	rank 0 dispatcher + N workers · non-blocking isend / iprobe	distributed memory · ≈ 2 µs per hop

MPI coordinates requests – CUDA does the GPU math.

Not chosen: OpenMP (CPU shared memory only) · NCCL (tensor collectives; we route requests) · HTTP only (kept as the baseline).

Tools, Technologies & Setup

Tools & Technologies

- **Hardware:** RTX 4060 Laptop (8 GB) · WSL2 Ubuntu · 16 cores
- **GPU programming:** CUDA 12.4, cuBLAS reference · Nsight Compute (pending)
- **Serving engine:** vLLM 0.11 + PyTorch 2.8 · our kernel as a torch custom op
- **Distributed:** Open MPI 5.0.10 + mpi4py · AWS multi-GPU (Terraform, not yet run)
- **Analysis:** NumPy / SciPy / pandas · paired tests · auto-generated report + dashboard

Experimental Setup

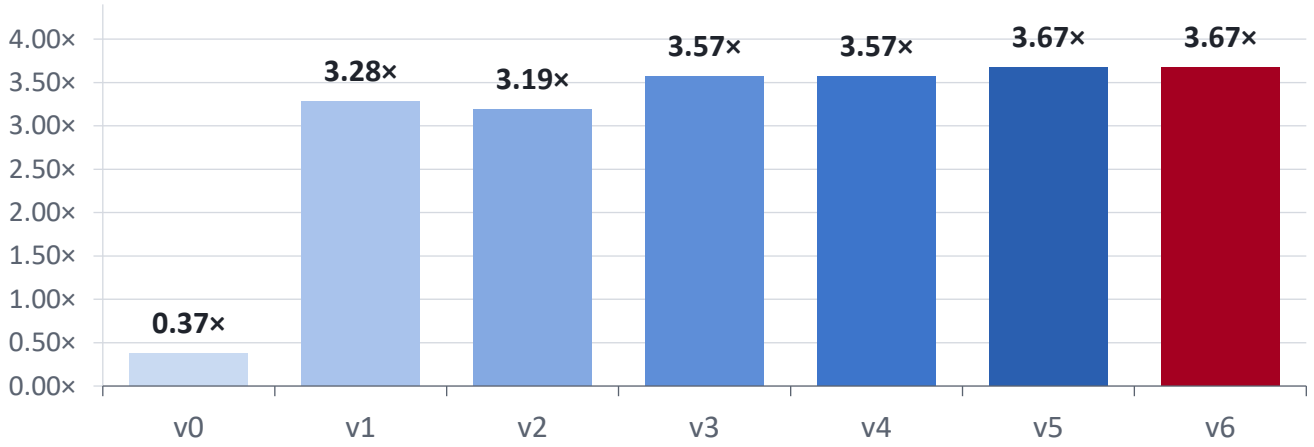
- **Kernel study:** 6 real layer shapes · int4, group 128 · 5 fresh runs · CPU-reference gate
- **Serving campaign:** 4 policies × 3 load patterns × 8 repeats = 96 runs (75 s each)
- **Calibrated load:** steady = 6.0 req/s (60% of measured capacity)
- **End-to-end in vLLM:** 5 configurations, 1–8 sequences · CUDA graphs · 3 repeats
- **Metrics:** TTFT, TPOT, p50/95/99, goodput, tokens/s, tokens/joule

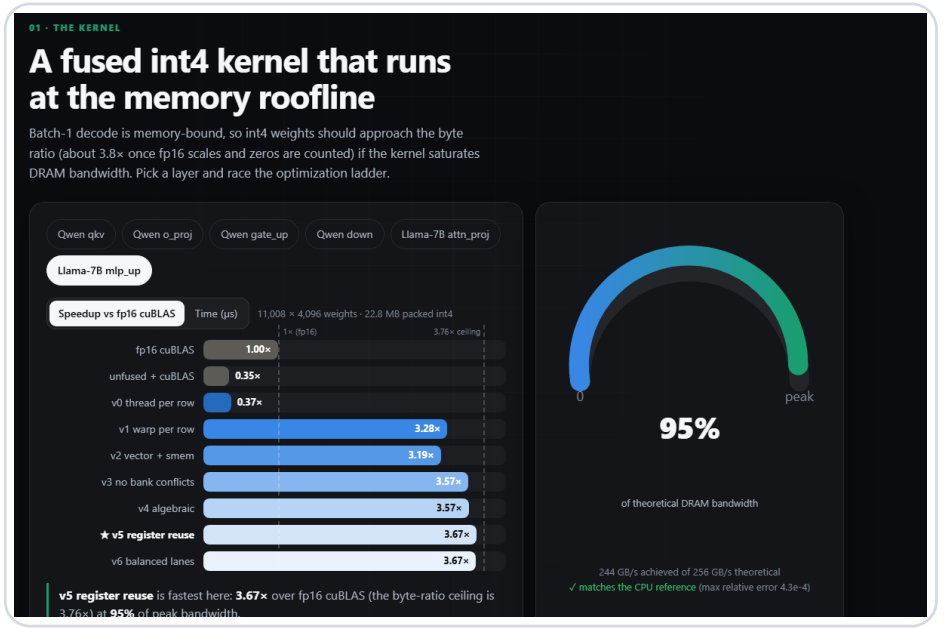
Every number is generated from raw logs by our own scripts – same seed in, same result out, every time.

Results: The CUDA Kernel

From a naive kernel to near the hardware limit — then verified inside a real serving engine.

Optimization ladder — speedup vs FP16 cuBLAS, batch 1, Llama-7B MLP layer





↑ from our live interactive dashboard

3.67× vs FP16 cuBLAS

95% of 256 GB/s bandwidth

Inside vLLM (real engine): 144 tokens/s at 1 sequence ≈ 1.01× Marlin's speed (parity, not a win). Marlin leads from 4+ sequences — next milestone: a tensor-core kernel.

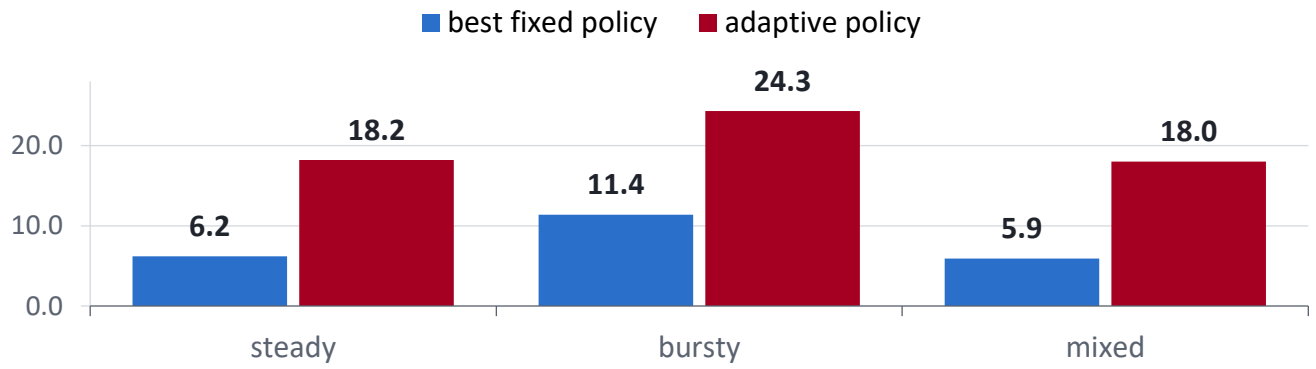
Speedup vs FP16 cuBLAS GEMV baseline, batch 1. Nsight Compute profiling PENDING (GPU counters blocked).

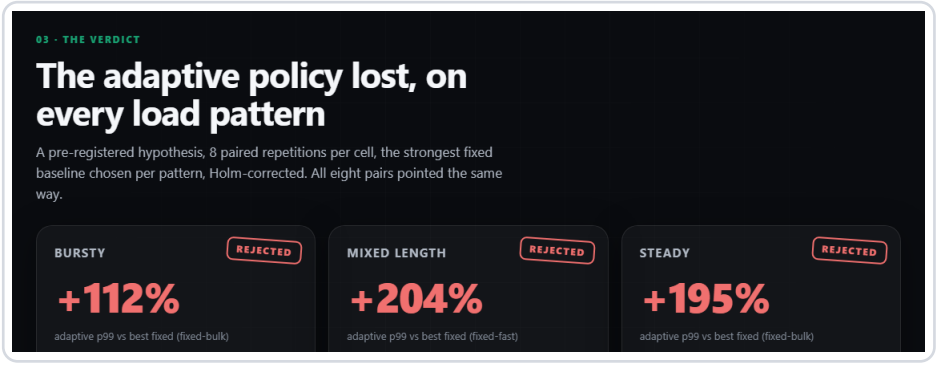
Results: Adaptive Routing & MPI

A pre-specified hypothesis, tested honestly — and a mechanism we found and explained

H1 rejected for a shared single GPU: p99 worsened 112–204% (Holm p = 0.023). Multi-GPU evaluation is PENDING.

p99 end-to-end latency (s): best fixed policy vs adaptive





↑ from our live interactive dashboard

112–204% worse p99

8 / 8 paired reps agree

Why? A shared GPU is time-sliced

2 replicas BOTH active → p99 ×2.1

2 replicas, ONE idle → +0.8% (free)

MPI ≈ 2 µs / hop

< 1% vs HTTP — not the bottleneck

What we learned:

not a failed idea — a precisely located one. Adaptive routing needs independent GPU capacity per replica; we found and explained exactly why it did not help here.

Setting: Qwen2.5-1.5B AWQ-int4, 2 replicas on ONE GPU · 96 paired runs (4 policies × 3 patterns × 8 reps) · MPI: Open MPI 5.0.10.

Challenges

Measurement traps

- Cache leak inflated bandwidth by ≈ 6% → fixed, re-run
- A library baseline was mis-measured → claim retracted
- Throughput metric hid saturation → redefined

Hardware limits

- Only 8 GB → a 1.5B-parameter model
- Both replicas share one GPU (0.47 scaling)
- Laptop clocks drift → randomized run order

Tooling

- GPU counters blocked by a Windows driver setting → Nsight profiling NOT yet done
- CUDA 12.4 toolkit vs PyTorch CUDA 12.8 build
- WSL2 disk / memory limits

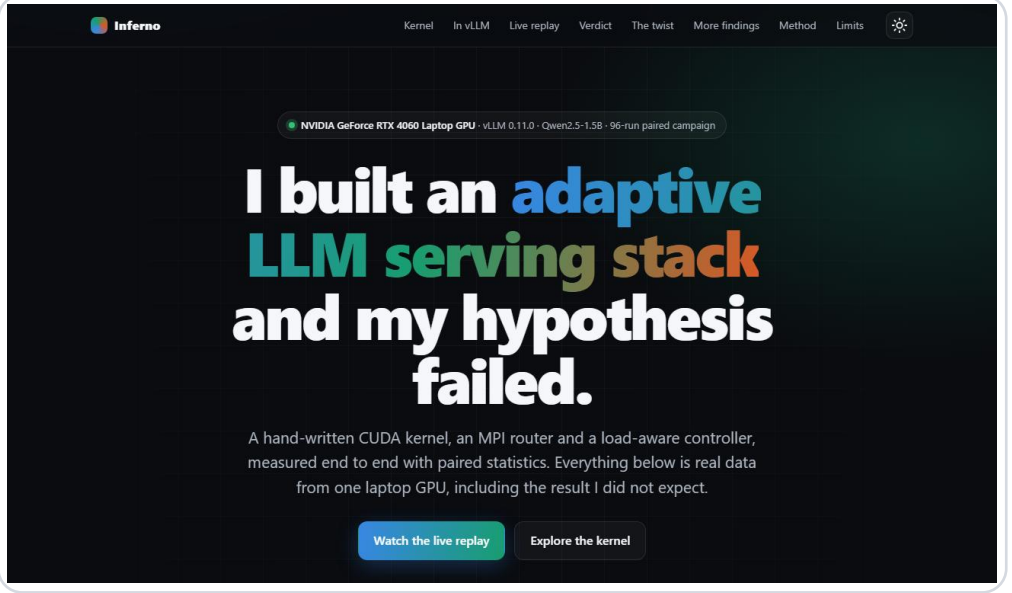
Engine limits

- vLLM cannot change batch size or precision live
- A restart costs ≈ 48 s
- → adaptivity implemented as routing across pre-loaded replicas

Contributions, Conclusions & Plan

Our contributions

- Fused int4 CUDA kernel: 87–95% of GPU memory bandwidth
- Integrated into vLLM — real end-to-end decode measured
- 96-run rigorous campaign, paired Holm-corrected statistics
- MPI-based distributed router with per-hop timing
- Explained WHY adaptive routing needs independent GPUs
- Live dashboard + 60-page report — 81 tests, reproducible



Explore: our live project dashboard

The plan to the final evaluation

NOW — MID-TERM

- Kernel (v0–v6), verified
- vLLM integration, measured
- 96-run campaign · MPI router

OCT – NOV 2026

- Nsight Compute profiling
- Tensor-core kernel (batch 8–64)
- Multi-GPU cloud + MPI setup

DEC 7–11, 2026

- Multi-GPU routing results
- Final report & dashboard
- FINAL EVALUATION

Team Work & Coordination

R1 Kernel & profiling

Likhith

DL.AI.U4AID24117

- CUDA kernel ladder v0–v6
- Small-batch sweep, Nsight

R2 Serving & statistics

Karthikeya

DL.AI.U4AID24020

- Open-loop client, calibration
- Paired tests, campaign runs

R3 MPI router & controller

Sandeep

DL.AI.U4AID24143

- Dispatcher / workers, policies
- Scaling and contention study

R4 Integration & documentation

A. Karthik

DL.AI.U4AID24104

- vLLM integration, cloud (Terraform)
- 60-page report, dashboard, deck

Coordination

weekly sync · shared Git repository · 81 automated tests passed

Documentation

60-page technical report · README · generated FINDINGS

Quality

every number generated from raw data · negative results kept

References:

1. G.-I. Yu et al., “Orca: A distributed serving system for transformer-based generative models,” in Proc. USENIX OSDI, 2022.
2. W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in Proc. ACM SOSP, 2023.
3. Y. Zhong et al., “DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving,” in Proc. USENIX OSDI, 2024.
4. P. Patel et al., “Splitwise: Efficient generative LLM inference using phase splitting,” in Proc. ACM/IEEE ISCA, 2024.
5. J. Lin et al., “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in Proc. MLSys, 2024.
6. E. Frantar et al., “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in Proc. ICLR, 2023.
7. E. Frantar et al., “MARLIN: Mixed-precision auto-regressive parallel inference on large language models,” in Proc. ACM PPoPP, 2025.
8. T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems,” in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2015.
9. A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in Proc. ACM OOPSLA, 2007, pp. 57–76.
10. S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” Commun. ACM, vol. 52, no. 4, pp. 65–76, 2009.

THANK YOU

Questions?